Moveit Trajectory Planning

Moveit Trajectory Planning
- 1. Usage Environment
- 2. Driving the Real Machine
  - 2.1 Starting the Real Device
  - 2.2 Starting MoveIt2
- 3. Trajectory Planning
  - 3.1. Visualization
  - 3.2. Launch Command
  - 3.3. Source Code Analysis

1. Usage Environment

ROS2: Humble

Enter the commands in the tutorial into the accompanying virtual machine to run the function.

2. Driving the Real Machine

Driving the real machine is achieved by subscribing to the Moveit2 `/joint_states` topic, converting the joint state information of the robotic arm into control data for the actual robotic arm.

Note: Since the real robotic arm does not have obstacle avoidance capabilities, some poses may encounter obstacles; therefore, the planned robotic arm movements should be as reasonable as possible and avoid locations with obstacles.

(It is recommended to use preset poses for demonstrating the driving of the real machine.)

The motherboard and virtual machine need to be configured for multi-machine communication. The robotic arm and virtual machine need to be on the same network, and the virtual machine's `ros_domain_id` must be consistent with the `ros_domain_id` in the Docker container. (Both devices are pre-configured with the same ros_domain_id at the factory; simply ensure the virtual machine and the robotic arm are on the same network.)

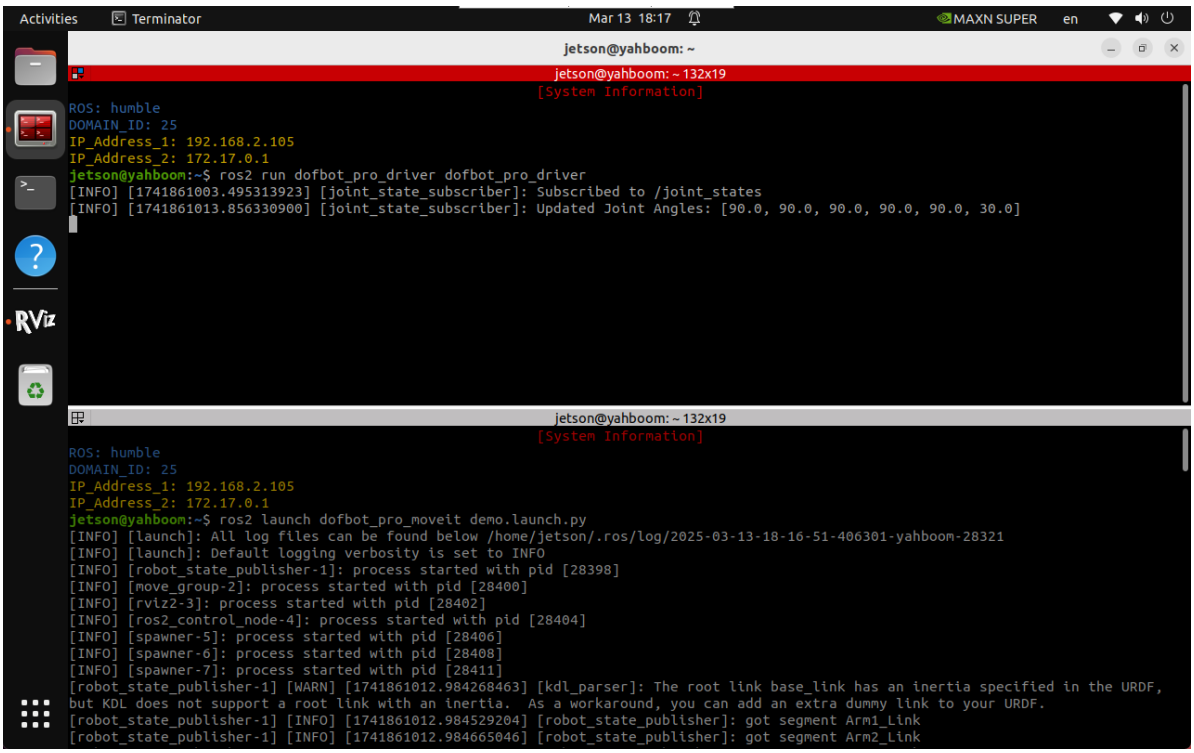
2.1 Starting the Real Device

```
#Start this command in Docker:
ros2 run dofbot_ctrl SimulateToArm
```

2.2 Starting MoveIt2

```
#Jetson-nano motherboard users need to start this command in the virtual machine;
Orin motherboard users can run this command directly on the motherboard.

ros2 launch dofbot_moveit demo.launch.py
```

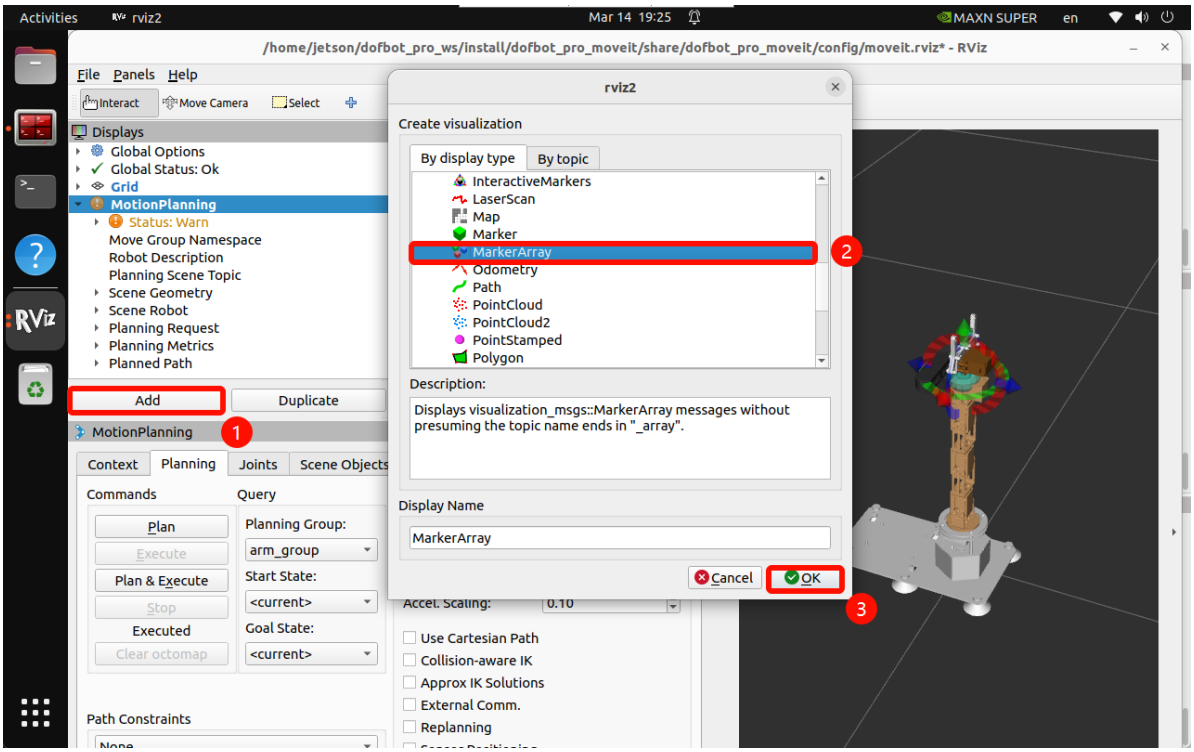


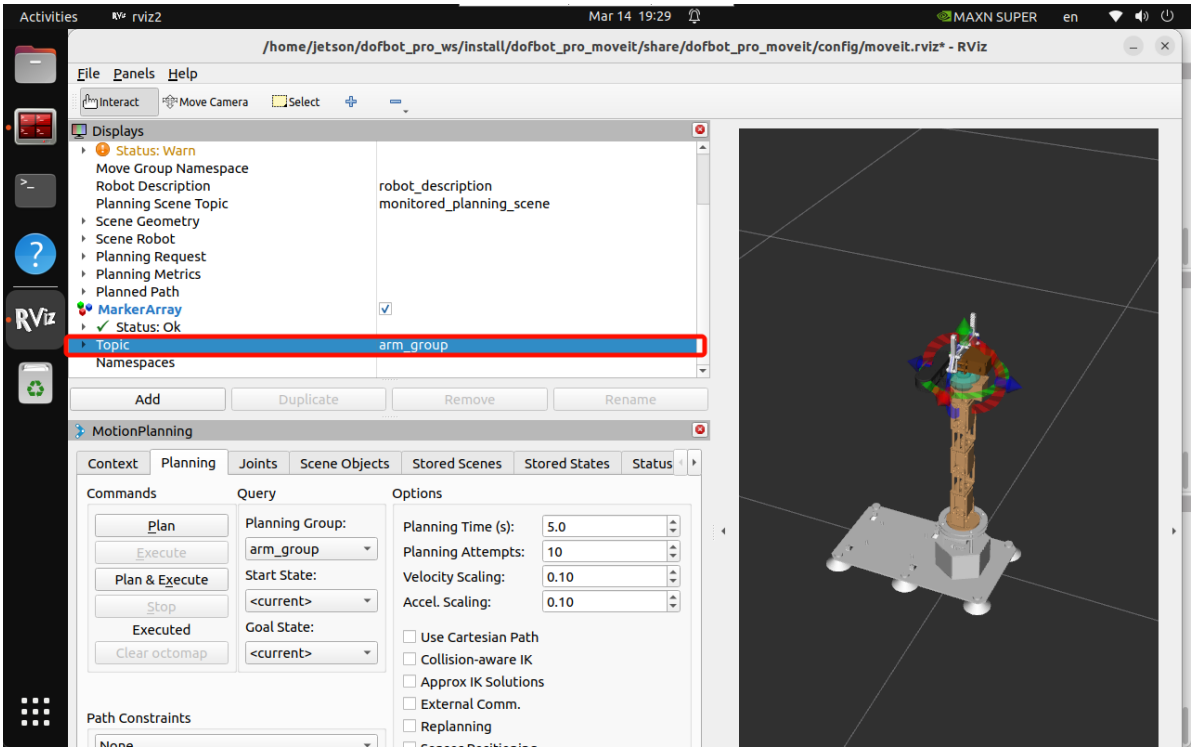
3. Trajectory Planning

After the program runs, the robotic arm will plan the trajectory to the target position, and the robotic arm's end effector will draw the motion trajectory in RViz2.

3.1. Visualization

Before executing the launch command, you must add the `MarkerArray` plugin in `RViz2` to display the planned path. For the `MarkerArray`, you need to select the `arm_group` topic.

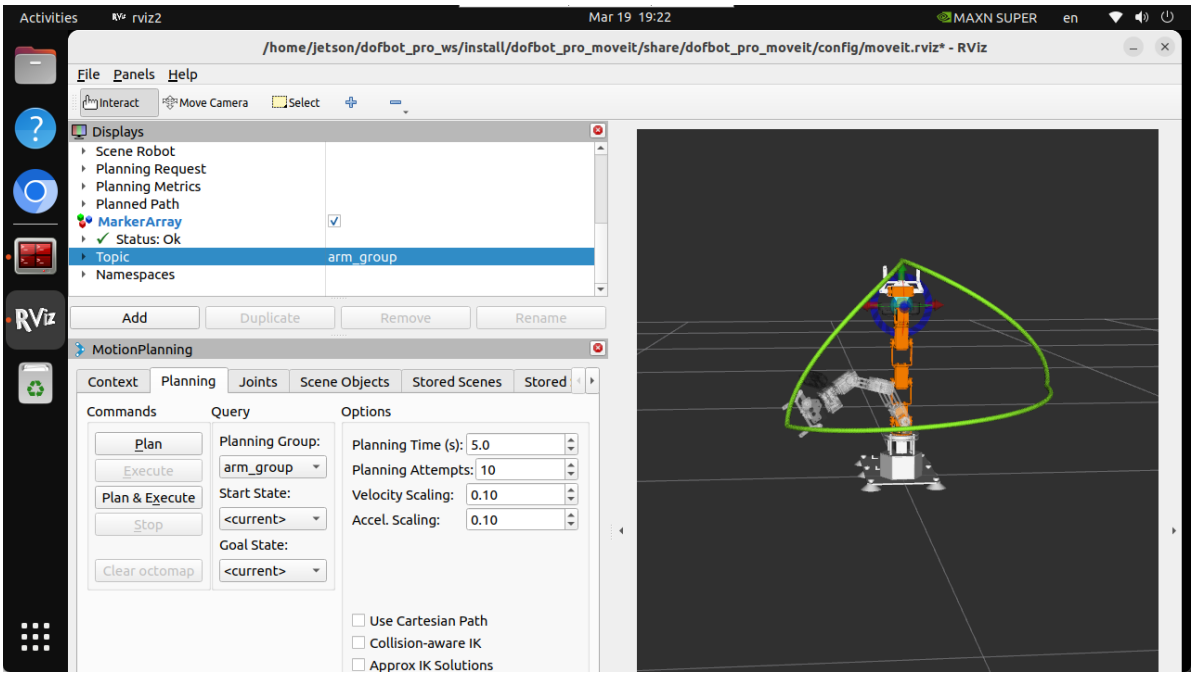




3.2. Launch Command

Ensure that the robotic arm has successfully loaded in MoveIt and that the message `You can start planning now!` appears. Then, run the following command: the robotic arm will automatically trace the trajectory of the planned path.

```
# Execute this command within the virtual machine
ros2 run dofbot_moveit multi_track_motion
```



3.3. Source Code Analysis

Source Code Path:

```
# Path within the virtual machine
~/dofbot_ws/src/dofbot_moveit/src/multi_track_motion.cpp
```

```
#include <rclcpp/rclcpp.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <geometry_msgs/msg/pose.hpp>
#include <random>
#include <moveit_visual_tools/moveit_visual_tools.h>
#include <moveit_msgs/msg/display_trajectory.hpp>
#include <vector>
#include <moveit/robot_trajectory/robot_trajectory.h>
#include <moveit/robot_state/robot_state.h>
#include <moveit/robot_model/robot_model.h>
#include <moveit/robot_model_loader/robot_model_loader.h>

class RandomMoveIt2Control : public rclcpp::Node
{
public:
  RandomMoveIt2Control()
    : Node("random_moveit2_control")
  {
   // Initialize other content
    RCLCPP_INFO(this->get_logger(), "Initializing RandomMoveIt2Control.");
  }

  void initialize()
  {
        // Load the robot model using RobotModelLoader
    robot_model_loader::RobotModelLoader
robot_model_loader(shared_from_this(),"robot_description");
    const moveit::core::RobotModelPtr& robot_model =
robot_model_loader.getModel();
    // Initialize move_group_interface_ and planning_scene_interface_ in this
function_
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>
(shared_from_this(), "arm_group");
    planning_scene_interface_ =
std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    const moveit::core::JointModelGroup* joint_model_group = robot_model-
>getJointModelGroup("arm_group");

    moveit_visual_tools::MoveItVisualTools visual_tools_(shared_from_this(),
"base_link", "arm_group",move_group_interface_->getRobotModel());
    move_group_interface_->setNumPlanningAttempts(10);  // Set the maximum
number of planning attempts to 10
    move_group_interface_->setPlanningTime(5.0);         // Set the maximum
planning time per attempt to 5 seconds.

    std::vector<double> target_joints = {1.57, -1.00, -0.61, 0.20, 0.0}; //
Target joint angles (units: radians)
        // // Move the robotic arm to the specified pose.
    move_group_interface_->setJointValueTarget(target_joints);

    // Plan Path
    moveit::planning_interface::MoveGroupInterface::Plan my_plan;
    bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
```

```
    if (success)
    {
              // Visualize the trajectory in RViz

 visual_tools_.publishTrajectoryLine(my_plan.trajectory_,move_group_interface_-
>getRobotModel()-
>getLinkModel("Gripping_point_Link"),joint_model_group,rviz_visual_tools::LIME_G
REEN);
      visual_tools_.trigger();
      RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
      moveit::core::MoveItErrorCode execute_result = move_group_interface_-
>execute(my_plan);
      if (execute_result == moveit::core::MoveItErrorCode::SUCCESS)
      {
        RCLCPP_INFO(this->get_logger(), "Trajectory executed successfully.");
        std::vector<double> target_joints = {0,     0,     0,     0,     0};
          // Move the robotic arm to the specified pose.
        move_group_interface_->setJointValueTarget(target_joints);
        moveit::planning_interface::MoveGroupInterface::Plan my_plan;
        bool success_two = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
        if (success_two)
        {
                  // Visualize the trajectory in RViz

visual_tools_.publishTrajectoryLine(my_plan.trajectory_,move_group_interface_-
>getRobotModel()-
>getLinkModel("Gripping_point_Link"),joint_model_group,rviz_visual_tools::LIME_G
REEN);
            visual_tools_.trigger();
            RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm
2nd.");
            moveit::core::MoveItErrorCode execute_result_2nd =
move_group_interface_->execute(my_plan);
            if (execute_result_2nd == moveit::core::MoveItErrorCode::SUCCESS)
            {
                RCLCPP_INFO(this->get_logger(), "Trajectory executed
successfully 2nd.");
                std::vector<double> target_joints = {-1.16, -0.50, -0.81, -0.79,
1.57};
                // Move the robotic arm to the specified pose.
                move_group_interface_->setJointValueTarget(target_joints);
                moveit::planning_interface::MoveGroupInterface::Plan my_plan;
                bool success_three = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
                if (success_three)
                {
                          // Visualize the trajectory in RViz

visual_tools_.publishTrajectoryLine(my_plan.trajectory_,move_group_interface_-
>getRobotModel()-
>getLinkModel("Gripping_point_Link"),joint_model_group,rviz_visual_tools::LIME_G
REEN);
                    visual_tools_.trigger();
                    RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving
the arm 3rd.");
```

```
                    moveit::core::MoveItErrorCode execute_result_3rd =
move_group_interface_->execute(my_plan);
                    if (execute_result_3rd ==
moveit::core::MoveItErrorCode::SUCCESS)
                    {
                        RCLCPP_INFO(this->get_logger(), "Trajectory executed
successfully 3rd.");
                    }
                }
            }
        }
      }
      else
      {
        RCLCPP_ERROR(this->get_logger(), "Trajectory execution failed with error
code: %d", execute_result.val);
      }
    }
    else
    {
      RCLCPP_ERROR(this->get_logger(), "Planning failed!");
    }



  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
  std::shared_ptr<moveit_visual_tools::MoveItVisualTools> visual_tools_;
};

int main(int argc, char **argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<RandomMoveIt2Control>();

  // Lazy Initialization
  node->initialize();

  rclcpp::spin(node);
  rclcpp::shutdown();
  return 0;
}
```